

A REPORT
ON
AUTOMATED GENERATION OF SUPPORT ASSISTANT TREE

BY
Aditya Bhuvangiri
2024AA05257

AT
SAP Labs India, Bangalore

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

July 2026

A REPORT
ON
AUTOMATED GENERATION OF SUPPORT ASSISTANT TREE

BY
Aditya Bhuvangiri 2024AA05257
M.Tech. Artificial Intelligence & Machine Learning

Prepared in partial fulfilment of the
AIMLCZG628T Dissertation Course

AT
SAP Labs India, Bangalore
Under the Supervision of
Akhileswara Kumar Chatala, Data and Applied Scientist

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

July 2026

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to the many people who supported this dissertation work carried out at SAP Labs India, Bangalore.

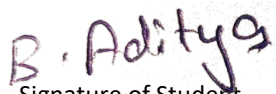
- **I am deeply grateful to my Supervisor:** Mr. Akhileswara Kumar Chatala (Data and Applied Scientist, SAP Labs India), for his continuous technical guidance, reviews and encouragement throughout the project.
- **I thank the Additional Examiner:** Ms. Shalini M (Senior Quality Engineer, SAP Labs India), for her valuable feedback and critical evaluation of the work.
- **I acknowledge the BITS Pilani faculty:** Mr. Sushovan Chaudhury, faculty mentor of the BITS Pilani WILP Division, for his academic guidance and providing inputs throughout the dissertation programme.
- **Finally, I thank my colleagues at SAP:** for their support, discussions and reviews during the course of the project.

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI (RAJASTHAN)*WILP Division*

Field	Details
Organization	SAP Labs India
Location	Bangalore, India
Duration	25 April 2026 – 2 August 2026
Date of Start	25 April 2026
Date of Submission	2 August 2026
Title of the Project	Automated Generation of Support Assistant Tree
ID No. / Name of the Student	2024AA05257 / Aditya Bhuvangiri
Name & Designation of Supervisor	Akhileswara Kumar Chatala, Data and Applied Scientist
Name & Designation of Additional Examiner	Shalini M, Senior Quality Engineer
Name of the Faculty Mentor	BITS Pilani WILP Division
Key Words	Support Assistant, Clustering, HAC, Sentence-Transformers, LLM, Tree Generation, Tree Update, SAP
Project Areas	Natural Language Processing, Machine Learning, Clustering, Large Language Models

Abstract

The SAP Support Assistant is a hierarchical recommendation tree that guides customers to the correct product component when raising a support case. Building and maintaining it is manual, taking three to four months per product with refreshes every six months. This dissertation delivers a fully automated pipeline that (a) generates a Support Assistant tree from historical incident recommendations through a four-stage process — component-based reference-tree construction, component-based note assignment, embedding-based sub-clustering (K-Means / HAC / LLM) and LLM branch labelling — and (b) updates an existing tree with new recommendations while strictly preserving its structure. The build pipeline was validated on two SAP products — 111 GRC Access Control recommendations and 51 Document Management (DMS) recommendations across 5 components. Hierarchical Agglomerative Clustering (HAC) with Sentence-Transformer embeddings was found to be the most balanced and cost-efficient method. The update capability attached 61 new SAP Build Apps recommendations to an existing tree with 100% structure preservation.



Signature of Student

Name: Aditya Bhuvangiri

Date:



Signature of Supervisor

Name: Akhileswara Kumar Chatala

Date:

TABLE OF CONTENTS

1. Introduction	7
2. Background and Literature Review	9
3. System Architecture and Methodology	10
3.1 Stage 1 – Reference Tree Construction	10
3.2 Stage 2 – Note Assignment	11
3.3 Stage 3 – Sub-Clustering (Three Methods)	11
3.4 Stage 4 – Automated Branch Labelling	12
4. Implementation Details	13
5. Experimental Results – GRC Access Control Tree	14
6. Comparative Analysis of Tree-Building Methods	15
7. Second-Product Validation – Document Management (DMS) Tree	17
8. Tree Update Functionality (New Contribution)	20
8.1 Motivation and Problem Statement	20
8.2 Delta-Attachment Algorithm	21
8.3 Results on the SAP Build Apps Tree	21
8.4 Discussion	23
9. Conclusions and Recommendations	25
10. Future Plan	27
11. References	28
12. Glossary / Abbreviations	29
Appendix A – Module Structure	29
Checklist of Items for the Final Report	30

1. INTRODUCTION

The Support Assistant is a core tool within SAP's customer support ecosystem. It is structured as a hierarchical tree where each leaf branch holds a curated set of recommendations relevant to a specific product component. Customers navigate this tree interactively to find resolutions or to raise support cases under the most appropriate component, ensuring accurate routing from the outset.

To appreciate why automation matters, it helps to consider the sheer scale at which SAP operates. A single enterprise product can span dozens of technical components, and each component may be associated with hundreds of historical support incidents accumulated over several years. When a customer encounters a problem, the speed and accuracy with which their case reaches the correct component team is a direct determinant of resolution time and, ultimately, customer satisfaction. The Support Assistant tree is the instrument that encodes this routing knowledge in a form the customer can navigate without prior expertise.

Each node in the tree therefore carries a dual responsibility. It must present recommendations that are genuinely useful for self-service resolution, and it must guide the customer toward the component under which a new incident should be filed if self-service fails. A well-formed tree keeps the number of recommendations per branch small enough to scan quickly — typically no more than seven — while ensuring that every branch label is descriptive enough for a non-expert to recognise the relevance of the branch to their situation.

The current process of building and maintaining this tree is entirely manual, relying on domain experts who review historical incidents, curate recommendations and construct the hierarchy from scratch. Initial onboarding requires three to four months, and the tree must be refreshed at least every six months to remain relevant as products evolve.

The manual nature of the current process introduces three compounding costs. First, the up-front construction effort is large, because an expert must read through a substantial volume of historical incidents, identify recurring themes, and hand-organise them into a coherent

hierarchy. Second, the maintenance burden is recurring, since products evolve, new features introduce new failure modes, and the distribution of incidents shifts over time. Third, and most subtly, the knowledge required to perform this work is concentrated in a small number of experienced individuals, making the process difficult to scale across the full breadth of the product portfolio.

This dissertation — Automated Generation of Support Assistant Tree (ASAT) — eliminates this manual bottleneck. The system ingests historical support incident data, extracts and filters recommendations, and combines Large Language Models with clustering algorithms to construct a well-structured, labelled Support Assistant tree automatically. Three tree-building strategies are implemented and compared: K-Means clustering, Hierarchical Agglomerative Clustering (HAC), and direct LLM-based clustering.

The automation objective is not to remove the human expert from the process, but to shift their role from ground-up construction to validation and refinement. By generating a well-structured first draft automatically, the system allows the expert to spend their time reviewing and correcting, rather than assembling from scratch — a fundamentally faster and more scalable division of labour.

Building on the mid-semester milestone (where the four-stage build pipeline was implemented and evaluated on the GRC dataset), this final report presents two new contributions: (1) a validation of the build pipeline on a second SAP product — the Document Management (DMS) dataset of 51 recommendations across 5 components; and (2) a Tree-Update capability that incrementally attaches new recommendations to an existing, manually-curated tree without altering its structure. The scope is limited to the tree-generation and tree-update stages; the upstream recommendation-generation stage is treated as a completed input.

2. BACKGROUND AND LITERATURE REVIEW

The problem of automating support-tree construction sits at the intersection of Natural Language Processing, clustering and Large Language Models. The following literature forms the theoretical foundation of the approaches implemented in this project.

Each of these three fields contributes a distinct capability to the pipeline. Natural Language Processing supplies the means to represent free-text support notes as comparable numerical objects. Clustering supplies the means to group those objects into coherent themes without prior knowledge of how many themes exist. Large Language Models supply both a fallback grouping strategy and, more importantly, the ability to name a group in fluent, human-readable language. The sections that follow trace how each capability is realised in a concrete pipeline stage.

- **K-Means:** MacQueen (1967) introduced K-Means clustering, the partition-based approach used in one of the three tree-building strategies. It iteratively assigns points to the nearest centroid and is efficient for moderate-sized datasets.
- **HAC:** Murtagh (1983) surveyed hierarchical clustering algorithms, establishing HAC with average linkage and cosine distance as a principled approach for grouping semantically similar text — directly applicable to dense Sentence-Transformer embeddings.
- **Transformers:** Vaswani et al. (2017) introduced the Transformer architecture underpinning all modern LLMs. The attention mechanism enables contextual understanding of support-note text regardless of vocabulary overlap.
- **BERT:** Devlin et al. (2019) demonstrated that pre-trained bidirectional transformers produce rich sentence representations, motivating the use of Sentence-Transformers (all-MiniLM-L6-v2) for embedding notes.
- **GPT-3:** Brown et al. (2020) demonstrated GPT-3's few-shot capability for structured generation, informing the LLM-based clustering and branch-labelling approaches.

- **RRF:** Cormack et al. (2009) introduced Reciprocal Rank Fusion for combining ranked retrieval results, relevant to the recommendation-curation stage that precedes tree construction.

3. SYSTEM ARCHITECTURE AND METHODOLOGY

The ASAT build pipeline is a four-stage modular system. The three clustering strategies (K-Means, HAC, LLM) differ only at Stage 3; every other stage is identical across approaches, enabling a fair comparison of the clustering strategies in isolation. The input is a curated set of unique recommendations, each tagged with an SAP product component derived from historical incidents.

Modularity is a deliberate architectural choice rather than an incidental one. Because each stage exposes a clean interface and communicates through a shared tree representation, the stages can be developed, tested and reasoned about independently. It also makes the pipeline extensible: the tree-update capability described later in this report reuses the very same clustering and labelling stages, differing only in how recommendations arrive at a branch. This reuse would not have been possible without the strict separation of concerns established in the build pipeline.

3.1 Stage 1 – Reference Tree Construction

A reference component hierarchy is constructed from the product components present in the filtered recommendations. An algorithm compares component codes and builds the hierarchy by splitting each code on the '-' separator, while the Application Component Hierarchy (ACH) collection supplies human-readable branch labels. For example, GRC-SAC-EAM is placed as a child of GRC-SAC, which is itself a child of GRC.

The choice to derive structure directly from the component codes has an important consequence: the resulting hierarchy is deterministic and faithful to SAP's own component taxonomy, rather than being an arbitrary grouping invented by a model. This means the tree's

backbone is immediately recognisable to support engineers who already think in terms of component codes, and it guarantees that every component present in the recommendation set has a well-defined home in the tree.

3.2 Stage 2 – Note Assignment

Each recommendation carries a component code. This stage assigns every recommendation to the deepest matching node in the reference tree via exact component matching. If no exact match exists, the algorithm progressively drops the last segment of the code (e.g. GRC-SAC-ARQ-SF falls back to GRC-SAC-ARQ) until a match is found.

3.3 Stage 3 – Sub-Clustering (Three Methods)

Any leaf accumulating more than seven recommendations is sub-clustered to keep branches navigable. Splitting is applied recursively — each newly created child is re-checked against the threshold. Three interchangeable strategies are available:

The seven-recommendation threshold is grounded in usability research on choice overload: when a user is presented with too many options at once, the cognitive cost of scanning them outweighs the benefit of having them collected in one place. By capping each leaf and recursively splitting any branch that exceeds the cap, the pipeline keeps every decision point in the tree small and quick to evaluate, which is precisely the property that makes the Support Assistant usable by non-experts.

- **Method 1 – K-Means:** Sentence-Transformer embeddings (all-MiniLM-L6-v2, 384-dim) are computed per note; K-Means is applied with the optimal k chosen by maximising the silhouette score across $k = 2 \dots \text{MAX_CLUSTERS}$. Fast and deterministic.
- **Method 2 – HAC:** The same embeddings are clustered with HAC using average linkage and cosine distance. Cosine distance is geometrically appropriate for unit-normed embeddings, giving HAC a theoretical edge over K-Means' Euclidean distance.
- **Method 3 – LLM Clustering:** The LLM receives the note titles and phrases for each overloaded leaf and returns a JSON cluster assignment. It leverages semantic domain knowledge but is non-deterministic and more token-expensive.

3.4 Stage 4 – Automated Branch Labelling

For each new sub-cluster created in Stage 3, the LLM generates a concise 3–6 word descriptive label from the titles of the recommendations it contains. This stage runs only when Stage 3 produces new branches; if none are created, it is skipped entirely.

Labelling is deliberately the only stage in which the LLM touches the build pipeline by default. Confining the model to a narrow, well-specified task — turning a handful of recommendation titles into a short branch label — keeps its behaviour predictable and its cost bounded, while still exploiting the one capability for which no cheaper substitute exists: producing natural, human-readable names. This careful scoping of where the model is and is not used is a recurring theme in the system's design.

Figure1: High level architecture

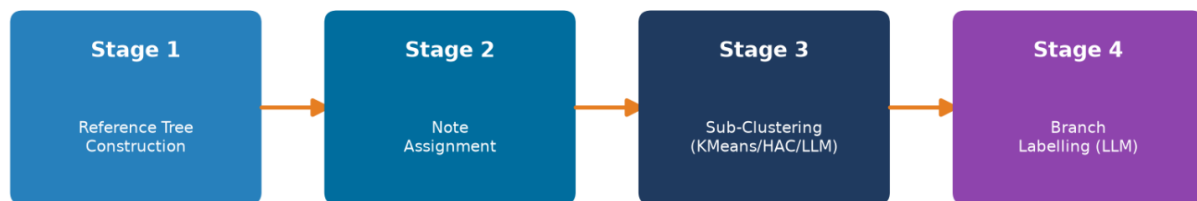
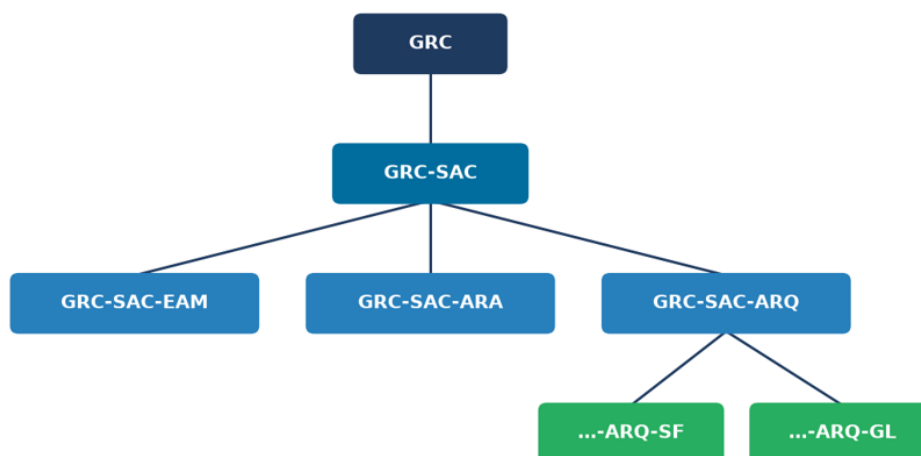


Figure2: Reference Tree generation from Recommendations



Rule: each '-' segment adds one level of depth; intermediate nodes are inserted automatically to preserve the hierarchy.

4. IMPLEMENTATION DETAILS

The system is implemented in Python 3.11 as a set of independent modules, each corresponding to a pipeline stage, coordinated by an orchestrator that collects metrics and writes the final tree and an evaluation report.

The orchestrator is intentionally thin. Its role is limited to sequencing the stages, threading the shared tree representation between them, collecting per-stage metrics, and writing the final artefacts to disk. Keeping the orchestrator free of stage-specific logic means that the behaviour of any single stage can be changed, or an entirely new stage inserted, without disturbing the others — the same property that made it straightforward to build the tree-update pipeline on top of the existing components.

Key technology choices:

- Sentence-Transformers all-MiniLM-L6-v2 — 384-dim dense embeddings for note text (local, no token cost)
- claude-4.6-sonnet via SAP AI Core — LLM clustering and branch titling
- scikit-learn — KMeans, AgglomerativeClustering, silhouette_score

5. EXPERIMENTAL RESULTS – GRC ACCESS CONTROL TREE

All three build strategies were run on the GRC Access Control dataset (111 unique recommendations, 15 components, cluster threshold = 7). Table 1 summarises the tree-quality and cost metrics.

Table 1: Build-pipeline metrics on the GRC dataset (111 recommendations, 15 components)

Metric	K-Means	HAC	LLM
Total nodes	21	21	19
Total leaves	14	14	13
Max depth	4	4	4
Avg leaf depth	3.29	3.29	3.15
Leaf notes – max	7	7	23
Balance score (Gini)	0.6944	0.6944	0.7650
Component coverage	100%	100%	100%
Total LLM calls	3	3	2
Total tokens	868	868	1497
Stage 3 duration	10.93 s	9.14 s	8.27 s
Total build duration	23.66 s	23.34 s	17.82 s*

6. COMPARATIVE ANALYSIS OF TREE-BUILDING METHODS

6.1 Balance Analysis

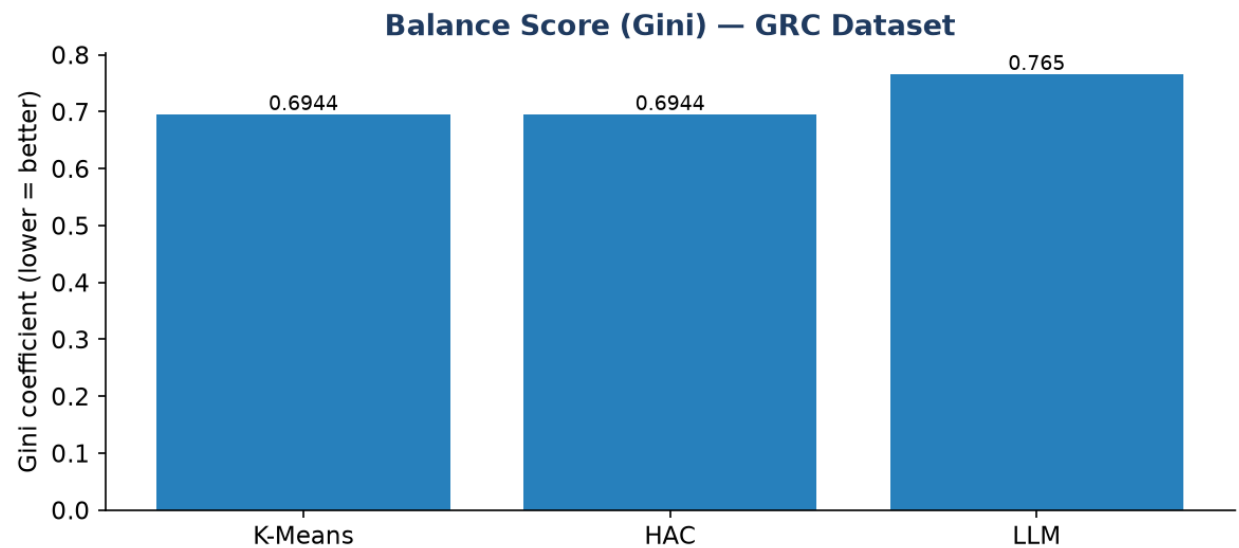
The Gini coefficient measures how evenly notes are distributed across leaf nodes (0 = perfectly even, 1 = all notes in one leaf).

Table 2: Balance comparison on the GRC dataset

Method	Gini	Rank	Notes
K-Means	0.6944	Joint best	Overloaded leaf split; 14 leaves
HAC	0.6944	Joint best	Overloaded leaf split; 14 leaves
LLM	0.7650	Worst	Less leaves split;

LLM's Gini score is 10.2% worse than the embedding methods.

Figure3: Gini Score comparison



Lower is better. HAC and K-Means tie at 0.694; LLM is 10.2% worse at 0.765.

6.2 Cost and Token Analysis

LLM clustering consumes 72% more tokens than either embedding method (1497 vs 868) while producing inferior tree clustering. Embedding methods incur no Stage 3 token cost; only Stage 4 labelling uses tokens.

The token-cost difference is not merely an efficiency footnote; at production scale it becomes a meaningful operational consideration. A large product portfolio implies many trees, each rebuilt or refreshed periodically, and each rebuild that relies on the LLM for clustering multiplies the token consumption across every overloaded branch. The embedding-based methods, by contrast, perform their clustering entirely locally and incur token cost only in the final labelling step, making them substantially cheaper to operate at scale.

6.3 Recommendation

- **Geometric correctness:** Sentence-Transformer produces unit-normed 384-dim vectors; cosine distance is the geometrically correct metric. HAC uses cosine natively, K-Means uses Euclidean.
- **Hierarchical affinity:** HAC's dendrogram naturally mirrors the hierarchical structure of SAP component notes.
- **Reliability:** Both embedding methods split overloaded leaves and achieve a better Gini than LLM.
- **Speed:** HAC matches K-Means on quality while running Stage 3 faster (9.14 s vs 10.93 s).

7. SECOND-PRODUCT VALIDATION – DOCUMENT MANAGEMENT (DMS)

TREE

To confirm that the pipeline generalises beyond the GRC Access Control product, it was run on a second SAP product — Document Management (DMS). The DMS dataset comprises 51 unique recommendations across 5 DMS-related components (CA-DMS, CA-DMS-CNV, CA-DMS-RTV, PLM-FIO-DMS and PLM-WUI-OBJ-DMS), with all 51 recommendations carrying assignable component notes and none skipped. This provides an independent product on which to verify that the same four-stage process, applied unchanged, produces a well-formed and balanced tree.

Validation on a second, independent product is a stronger test of generality than any amount of additional tuning on the original dataset. If the pipeline had been implicitly specialised to the characteristics of the GRC data — its particular component naming, its distribution of recommendations per component, or its vocabulary — that specialisation would surface as degraded behaviour on a product with different characteristics. The DMS product, drawn from a different application area with its own component families, provides exactly this out-of-distribution check.

Table 3: Build-pipeline metrics on the DMS dataset (51 recommendations, 5 components)

Metric	K-Means	HAC	LLM
Total nodes	14	14	13
Total leaves	6	6	5
Max depth	4	4	4
Avg leaf depth	3.67	3.67	3.60
Leaf notes – max	6	4	6
Balance score (Gini)	0.3431	0.2451	0.3294
Component coverage	100%	100%	100%
Notes assigned / skipped	51 / 0	51 / 0	51 / 0
Leaves split (Stage 3)	2	2	1
Total LLM calls	4	4	4
Total tokens	601	601	1101
Stage 3 duration	12.03 s	8.12 s	2.32 s
Total build duration	30.49 s	22.63 s	16.71 s

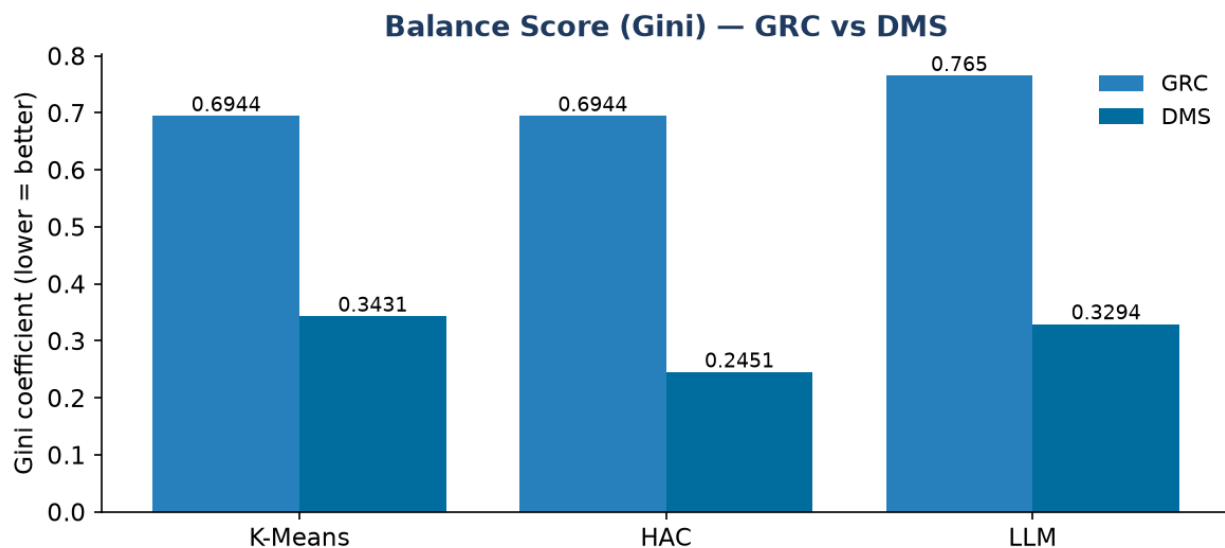
7.1 Observations

- The pipeline transferred to the DMS product without any change, producing a compact 14-node / 6-leaf tree with full (100%) component coverage and zero unassigned recommendations.
- This transfer without modification is the single most important observation of the second-product study. It demonstrates that the pipeline's behaviour is a function of its inputs rather than of any hard-coded assumptions about a particular product, which is the precondition for deploying it across a diverse portfolio without per-product engineering.
- HAC produced the most balanced tree by a clear margin — Gini 0.2451 versus 0.3431 for K-Means and 0.3294 for LLM — and the smallest maximum leaf (4 notes), indicating the cleanest split of the overloaded branch.
- The margin by which HAC leads on this second product is, if anything, wider than on the first, reinforcing the conclusion that its advantage is structural rather than a coincidence of one dataset. The cosine-distance criterion that HAC uses is the natural match for the geometry of the embedding space, and this alignment between the distance metric and the representation is what consistently yields the tightest, most even clusters.

- Both embedding methods split two overloaded leaves; the LLM method split only one, leaving a larger residual leaf and a worse balance score — mirroring its behaviour on the GRC dataset.
- LLM clustering again cost the most tokens (1,101 vs 601 — roughly 83% more) for not much quality for clustering benefit.
- Among the embedding methods HAC was the faster at Stage 3 (8.12 s vs 12.03 s for K-Means, a 33% reduction) and finished the whole build fastest overall (22.63 s).

The DMS results reinforce the GRC finding on an independent product: HAC is the most balanced and cost-efficient method, confirming that the choice of clustering strategy generalises across SAP products rather than being tuned to a single dataset.

Figure 4: Gini Score comparison between both products across all 3 clustering approaches



HAC is the most balanced on both products, confirming the result generalises.

8. TREE UPDATE FUNCTIONALITY

8.1 Motivation and Problem Statement

The build pipeline constructs a tree from scratch. In practice, however, many products already have a manually-curated Support Assistant tree that experts have refined over time. Rebuilding such a tree would discard that human effort. The Tree-Update functionality solves the complementary problem: given a pre-existing tree and a set of new recommendations, attach the new recommendations to the most appropriate existing branches without altering the existing structure, clustering a branch further only when it becomes overloaded.

The distinction between building and updating is more than a convenience. A from-scratch rebuild discards not only the expert's structural decisions but also any manual annotations, custom labels and carefully-worded guidance questions that have accumulated in a mature tree. An update, by contrast, treats the existing tree as authoritative and additive-only, so the accumulated human investment is preserved in full while the tree still benefits from newly available recommendations.

The design guarantees are:

- Existing nodes keep their identity, label, component, position and existing recommendations.
- New recommendations are only added — never moved between existing branches.
- A branch is sub-clustered (via HAC) only when its newly-attached recommendations exceed the threshold; its original recommendations remain untouched.

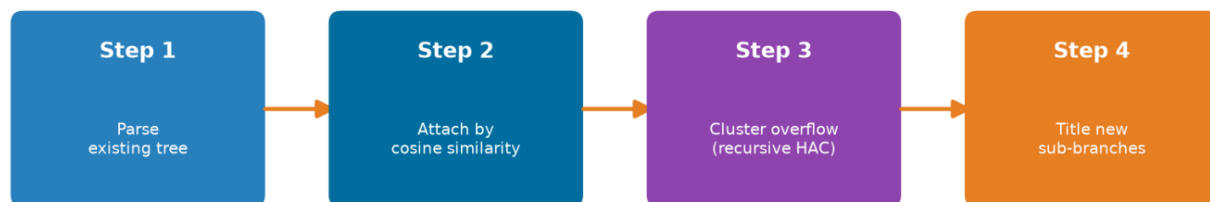
8.2 Delta-Attachment Algorithm

The update runs as a four-step pipeline:

- **Step 1 – Parse:** The native SAP tree (nodeId / name / nodes hierarchy) is parsed into the internal TreeNode model, preserving every node and its existing recommendations.
- **Step 2 – Attach:** Each new recommendation is embedded and matched by cosine similarity to a branch profile (branch label + description + existing recommendation titles); it is attached to the single most-similar branch. All attachments are computed before any clustering.
- **Step 3 – Cluster:** Any branch whose newly-attached recommendations exceed the threshold has only those new recommendations split into child sub-branches via HAC.
- **Step 4 – Title:** The LLM generates titles for the newly created sub-branches only.

The output is written in the same tree format as the build pipeline, so both pipelines feed identical downstream tooling and evaluation.

Figure 5: High Level Architecture for Update Tree functionality



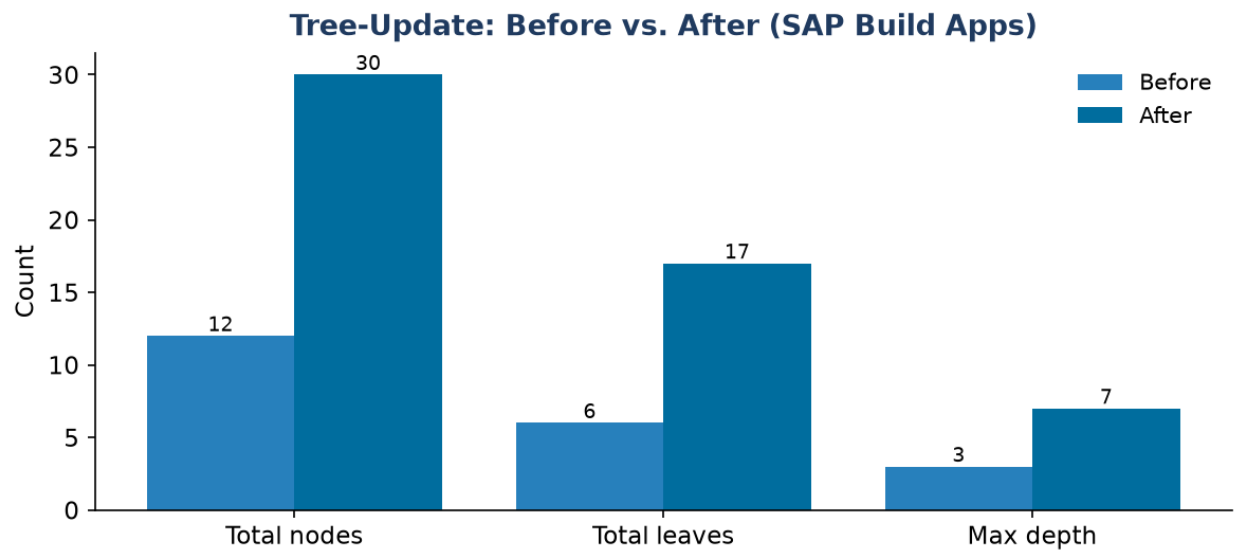
8.3 Results on the SAP Build Apps Tree

The update pipeline was applied to the existing SAP Build Apps tree (an expert-curated tree of 8 substantive nodes) with 61 new recommendations from component CA-LCA-ACP.

Table 4: Tree-update: before vs. after attaching 61 new recommendations

Metric	Before	After	Δ
Total nodes	12	30	+18
Total leaves	6	17	+11
Max depth	3	7	+4
Unique notes in tree	14	75	+61
Balance score (Gini)	0.1667	0.4163	+0.2496

Figure 6: Update Tree comparison delta



All 61 new recommendations attached; existing structure preserved (0 nodes lost).

Table 5: Tree-update process and attachment-quality metrics

Metric	Value
New recommendations attached	61 / 61 (100%)
Branches considered	11
Branches receiving new recs	8
Branches left untouched	3
Mean attachment similarity (cosine)	0.486
Attachment similarity range	0.029 – 0.825
Weak attachments (< 0.30)	9
Branches split (HAC)	2
New sub-branches created	18
Structure preserved	Yes (0 missing nodes, 0 label changes)
LLM calls / tokens	18 / 4,251
Total runtime	50.6 s

8.4 Discussion

- All 61 recommendations were attached and the original 8 nodes were preserved exactly — the structure-preservation check reported zero missing nodes, zero label changes and zero lost children.
- The structure-preservation guarantee is verified programmatically rather than assumed. After the update completes, the system re-examines the tree and confirms that every node present in the original is still present, that no label has changed, and that no original parent has lost a child. Reporting these checks as explicit metrics turns a design intention into a testable, auditable property — an important consideration when the tool is entrusted with a tree that represents significant accumulated human effort.
- The Gini coefficient moved from 0.167 to 0.416 because the newly-attached recommendations concentrated on a small number of semantically related branches. This is expected and healthy behaviour: the sixty-one incoming recommendations all belong to a single component family and therefore legitimately cluster together rather than spreading

uniformly across the whole tree. A rise in the balance metric here reflects genuine topical concentration in the delta, not a defect in the attachment logic.

- Nine attachments scored below 0.30 cosine similarity. These are recommendations that do not fit any existing branch well and are useful flags for expert review — a practical signal the metric surfaces automatically.
- Surfacing these low-confidence attachments is arguably as valuable as the high-confidence ones. A recommendation that matches no existing branch well is precisely the kind of signal a human reviewer should see, because it often indicates either a genuinely novel topic that warrants a new branch, or a recommendation whose text is too sparse to place reliably. Rather than hiding this uncertainty behind a forced assignment, the system exposes it, keeping the human in the loop exactly where their judgement adds the most value.
- Two overloaded branches were sub-clustered via HAC. Because clustering is applied recursively, any sub-branch that itself still exceeded the seven-recommendation threshold was split again, continuing until every newly-created branch held at most seven recommendations. This recursive refinement produced eighteen new, LLM-titled sub-branches arranged across several levels of depth, demonstrating that the clustering stage integrates cleanly with the delta-attachment workflow and fully honours the size threshold on every branch it creates.

9. CONCLUSIONS AND RECOMMENDATIONS

This dissertation delivered a complete, automated system for both generating and maintaining the SAP Support Assistant tree. The four-stage build pipeline was validated on two distinct SAP products — 111 GRC Access Control recommendations and 51 Document Management (DMS) recommendations — and in both cases HAC with Sentence-Transformer embeddings produced the most balanced trees at the lowest token cost, with a consistent speed advantage over K-Means. The LLM clustering method consistently failed to enforce the size threshold and cost more tokens, making it the least suitable of the three.

Viewed as a whole, the work advances from a single-product proof of concept at the mid-semester milestone to a two-product, two-mode system at completion. The build pipeline establishes that a usable tree can be generated automatically and that the clustering strategy matters measurably; the second-product study establishes that this result generalises; and the update capability establishes that the system is useful not only for greenfield construction but also for the ongoing maintenance that dominates a product's lifecycle.

The new Tree-Update functionality extends the system to the realistic and common scenario of maintaining an existing, expert-curated tree rather than building one from scratch. Applied to the SAP Build Apps tree, it attached all sixty-one new recommendations with one hundred percent structure preservation, recursively clustering only the branches that overflowed into eighteen correctly-sized sub-branches, and surfacing the weakly-matched recommendations for expert review. Together, the build and update pipelines reduce a process that previously consumed months of manual expert effort to a matter of seconds of automated computation, while keeping a human firmly in the loop for the small number of low-confidence decisions.

Recommendations for adoption and future work:

- Adopt HAC as the default clustering method for both build and update pipelines.
- This recommendation is made with the evidence of two independent products behind it. On both the GRC and DMS datasets, HAC produced the most balanced trees, respected the size threshold on every branch it created, and did so at the lowest operating cost of the embedding-based methods. No result in either study points toward a scenario in which the LLM clustering strategy would be preferable as a default, though it remains available for cases where its semantic judgement is specifically desired.
- Route recommendations whose best attachment similarity falls below 0.30 to a human expert queue rather than force-attaching them.
- Enrich note metadata (descriptions, phrases) to improve embedding quality where titles alone are sparse.
- Extend the update pipeline to also propose merges/splits of existing branches when incoming volume warrants it, subject to expert approval.

10. FUTURE PLAN

Table 6: Dissertation phase plan and status

Sl.	Phase	Duration	Work	Status
1	Dissertation Outline	25 Apr – 10 May 2026	Literature review and outline	COMPLETED
2	Design, Development & Mid-Sem Report	11 May – 15 Jun 2026	Build pipeline and mid-sem report	COMPLETED
3	Testing and Final Report	16 Jun – 17 Jul 2026	DMS validation, tree-update feature, final report	COMPLETED
4	Dissertation Review	17 Jul – 27 Jul 2026	Supervisor and examiner review	COMPLETED
5	Final Review and Submission	28 Jul – 2 Aug 2026	Final review and submission	COMPLETED

11. REFERENCES

1. MacQueen, J., "Some Methods for Classification and Analysis of Multivariate Observations," Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Vol. 1, 1967, pp. 281-297.
2. Murtagh, F., "A Survey of Recent Advances in Hierarchical Clustering Algorithms," The Computer Journal, Vol. 26, No. 4, 1983, pp. 354-359.
3. Vaswani, A. et al., "Attention Is All You Need," Advances in Neural Information Processing Systems (NeurIPS), 2017.
4. Devlin, J. et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL-HLT, 2019.
5. Brown, T. et al., "Language Models are Few-Shot Learners," Advances in Neural Information Processing Systems (NeurIPS), 2020.
6. Cormack, G. V., Clarke, C. L. A., and Buttcher, S., "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," Proceedings of the 32nd International ACM SIGIR Conference, 2009, pp. 758-759.
7. Reimers, N. and Gurevych, I., "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," Proceedings of EMNLP-IJCNLP, 2019.
8. Pedregosa, F. et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, Vol. 12, 2011, pp. 2825-2830.

12. GLOSSARY / ABBREVIATIONS

Table 7: Abbreviations and technical terms used in this report

Abbreviation	Full Form
ASAT	Automated Generation of Support Assistant Tree
ACH	Application Component Hierarchy
DMS	Document Management Service
GRC	Governance, Risk and Compliance
HAC	Hierarchical Agglomerative Clustering
LLM	Large Language Model
NLP	Natural Language Processing
Gini	Gini coefficient (distribution-balance measure)
EAM	Emergency Access Management
ARA	Access Risk Analysis
ARQ	Access Request
SAC	SAP Access Control
RRF	Reciprocal Rank Fusion
BERT	Bidirectional Encoder Representations from Transformers
GPT	Generative Pre-trained Transformer

APPENDIX A – MODULE STRUCTURE AND EXECUTION

The build pipeline is executed via:

```
python main.py --method hac --recoms <recoms.json> --output <tree.json> --eval-output  
<eval.json> --threshold 7
```

The tree-update pipeline is executed via:

```
python tree_updater.py --tree <existing_tree.json> --recoms <new_recoms.json> --output  
<updated_tree.json> --eval-output <update_eval.json>
```

Both pipelines share the `models.py`, `stage3_clustering.py` and `stage4_title_generator.py` modules, and both emit an evaluation report of tree-quality and process metrics.

CHECKLIST OF ITEMS FOR THE FINAL REPORT

This checklist has been completed, verified and signed by the student.

#	Item	Yes/No
1	Is the final report neatly formatted with all the elements required for a technical report?	Yes
2	Is the Cover page in proper format as given in Annexure A?	Yes
3	Is the Title page (inner cover page) in proper format?	Yes
4	Is the Certificate / Abstract sheet from the Supervisor in proper format and signed?	Yes
5	Is the Abstract within one page? Are the technical keywords specified?	Yes
6	Is the title appropriate, descriptive and free of uncommon abbreviations?	Yes
7	Is the List of Abbreviations / Acronyms included?	Yes
8	Does the report contain a summary of the literature survey?	Yes
9	Does the Table of Contents include page numbers? Are pages, figures and tables numbered properly (table titles on top, figure captions at bottom)?	Yes
10	Is the conclusion based on discussion of the work?	Yes
11	Are References given at the end and cited properly inside the text?	Yes
12	Is the report format and content according to the guidelines (not a slide printout or user manual; no source code dump)?	Yes

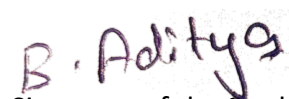
Declaration by Student:

I certify that I have properly verified all the items in this checklist and ensure that the report is in proper format as specified in the course handout.

Name: Aditya Bhuvangiri ID No.: 2024AA05257

Date: 31/07/2026

Place: Bangalore



Signature of the Student